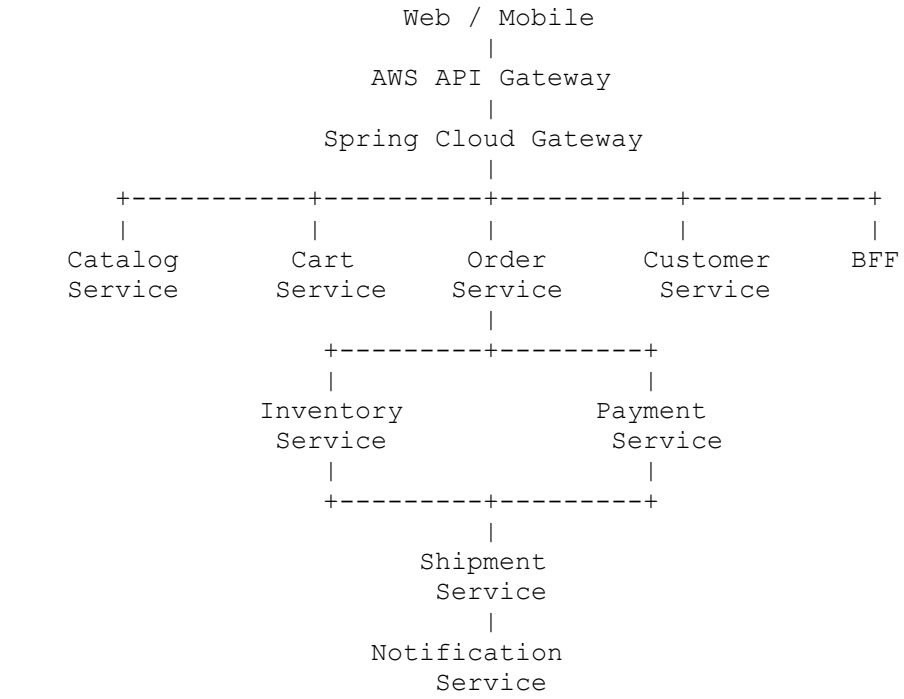
True Saya Practical Hands on Project

I reviewed the attached List od Microservices Design Patterns and Principles.docx. It contains **14 pattern groups** covering decomposition, communication, service discovery, data management, resilience, transactions, deployment, observability, security, infrastructure, Kubernetes/cloud-native architecture, workflows, caching, and emerging distributed-system patterns. [List od Mi...iples.docx | Word]

Below is a **step-by-step implementation blueprint**, mapped to one enterprise **E-Commerce Platform** using **Java + Spring Boot + Spring Web + Spring Data JPA/Hibernate + REST APIs + Kafka/AWS messaging + PostgreSQL + Redis + Docker + Kubernetes/EKS + AWS**. The emphasis is on how you would actually implement the patterns, not merely define them.

0. Start with the target architecture

Use these Spring Boot microservices:



For every service, use a conventional layered structure:

```
order-service
```

```
├── controller
├── application
├── domain
├── repository
├── integration
├── messaging
├── configuration
├── exception
└── observability
```

Recommended Spring dependencies:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-validation
spring-boot-starter-security
spring-boot-starter-oauth2-resource-server
spring-boot-starter-actuator
spring-kafka
resilience4j-spring-boot3
micrometer-registry-prometheus
opentelemetry
postgresql
flyway-core
```

---

1. Decomposition Patterns

The document lists Business Capability, DDD Bounded Context, Single Responsibility Service, Strangler Fig, Modular Monolith First, Self-Contained Services and DDD. [List od Mi...iples.docx | Word]

Step 1: Identify business capabilities

Start with business functionality rather than database tables:

```
Product Management  -> Catalog Service
Shopping Cart       -> Cart Service
Ordering            -> Order Service
Payment             -> Payment Service
Stock               -> Inventory Service
Shipping            -> Shipment Service
Customer            -> Customer Service
Notifications       -> Notification Service
```

Principle

Do **not** decompose as:

```
Controller Microservice
```

```
DAO Microservice
Validation Microservice
```

Decompose around business capabilities.

Step 2: Establish DDD bounded contexts

Define business boundaries:

```
Ordering Context
 ├── Order
 ├── OrderItem
 └── OrderStatus

Payment Context
 ├── Payment
 ├── PaymentMethod
 └── Refund

Inventory Context
 ├── Stock
 └── Reservation

Shipping Context
 ├── Shipment
 └── Tracking
```

Treat each bounded context as an independent domain model.

Step 3: Implement aggregate roots

Example Order aggregate:

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    private UUID id;

    private UUID customerId;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    @OneToMany(
        mappedBy = "order",
```

```
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<OrderItem> items = new ArrayList<>();

    public void confirm() {
        this.status = OrderStatus.CONFIRMED;
    }

    public void cancel() {
        this.status = OrderStatus.CANCELLED;
    }
}
```

Keep business rules inside the domain/aggregate rather than spreading them across REST controllers.

Step 4: Enforce single responsibility

`PaymentService` should handle:

```
Authorize
Capture
Refund
Payment status
```

It should not contain:

```
Stock reservation
Product pricing
Shipping
Customer registration
```

Step 5: Apply Self-Contained Service

Every service owns:

```
API
Business Rules
Domain Model
Database
Configuration
Deployment
Monitoring
```

Example:

```
order-service
    |
    +-- REST API
    +-- Order Domain
    +-- Spring Data JPA
    +-- Order PostgreSQL DB
    +-- Kafka Producer/Consumer
    +-- Docker Image
    +-- Kubernetes Deployment
```

Step 6: Use Strangler Fig for legacy migration

If migrating an existing monolith:

```
Step 1 -> Put Gateway before monolith
Step 2 -> Extract Catalog
Step 3 -> Route /products/** to Catalog Service
Step 4 -> Extract Customer
Step 5 -> Extract Order
Step 6 -> Extract Inventory
Step 7 -> Extract Payment
Step 8 -> Retire equivalent monolith modules
```

At intermediate stages:

```
                 Gateway
                 /     \
                /       \
     New Microservices   Old Monolith
```

This avoids a high-risk all-at-once rewrite.

---

2. API Gateway Pattern

API Gateway and BFF are among the key communication patterns in the attached document.
[List od Mi...iples.docx | Word]

Step 1: Create Spring Cloud Gateway

```
spring:
  cloud:
    gateway:
      routes:

        - id: order-service
          uri: http://order-service
```

```
          predicates:
            - Path=/api/orders/**

        - id: catalog-service
          uri: http://catalog-service
          predicates:
            - Path=/api/products/**

        - id: customer-service
          uri: http://customer-service
          predicates:
            - Path=/api/customers/**
```

Production architecture:

```
Internet
   |
AWS WAF
   |
AWS ALB/API Gateway
   |
Spring Cloud Gateway
   |
Kubernetes Services
```

Gateway responsibilities:

```
Authentication
Authorization
Routing
Rate limiting
Correlation IDs
Request logging
CORS
API policy enforcement
```

Do **not** put core Order/Payment business logic inside the gateway.

---

3. Backend for Frontend Pattern

Implement separate experience APIs:

```
Web Client ------> Web BFF ------\
                                  \
                                   Microservices
                                  /
Mobile Client ---> Mobile BFF ---/
```

Example:

```
@RestController
@RequestMapping("/mobile/products")
class MobileProductController {

    private final ProductAggregationService service;

    @GetMapping("/{id}")
    MobileProductResponse getProduct(@PathVariable UUID id) {
        return service.getMobileProduct(id);
    }
}
```

Mobile BFF could return:

```
{
  "name": "Laptop",
  "price": 79999,
  "thumbnail": "...",
  "available": true
}
```

while the web BFF can return richer product information.

---

4. Service-to-Service REST

For calls requiring an immediate result:

```
Order Service
      |
      | GET inventory
      v
Inventory Service
```

With modern Spring HTTP interfaces:

```
@HttpExchange("/api/inventory")
public interface InventoryClient {

    @GetExchange("/{sku}")
    InventoryResponse getInventory(
        @PathVariable String sku);
}
```

Controllers should expose resource-oriented endpoints:

```
GET    /api/products/{id}
POST   /api/orders
GET    /api/orders/{id}
POST   /api/payments
```

```
POST   /api/inventory/reservations
DELETE /api/inventory/reservations/{id}
```

Avoid deep synchronous chains:

```
Order -> Inventory -> Payment -> Shipping -> Notification
```

Prefer event-driven processing for long-running workflows.

---

5. API Composition / Aggregator

Suppose the storefront needs:

```
Product
Price
Inventory
Reviews
```

Implement:

```
Product Aggregator
       |
       +--> Catalog Service
       +--> Pricing Service
       +--> Inventory Service
       +--> Review Service
```

Return:

```
{
   "productId": "P100",
   "name": "Laptop",
   "price": 79999,
   "stock": 24,
   "rating": 4.5
}
```

Keep fan-out bounded because every downstream synchronous dependency adds latency and another failure point.

---

6. Event-Driven Architecture

The document includes Event-Driven Architecture, Publish-Subscribe, Event Notification, Event-Carried State Transfer, Domain Events and Message Broker. [List od Mi...iples.docx | Word]

Step 1: Define business events

```
public record OrderCreatedEvent(
        UUID eventId,
        UUID orderId,
        UUID customerId,
        BigDecimal amount,
        Instant occurredAt) {
}
```

Use names representing completed business facts:

```
OrderCreated
InventoryReserved
InventoryRejected
PaymentAuthorized
PaymentFailed
ShipmentCreated
OrderConfirmed
OrderCancelled
```

Step 2: Publish through Kafka

```
@Service
@RequiredArgsConstructor
public class OrderEventPublisher {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void publish(OrderCreatedEvent event) {
        kafkaTemplate.send(
            "ecommerce.orders",
            event.orderId().toString(),
            event
        );
    }
}
```

On AWS, Kafka can be provided through **Amazon MSK**.

For simpler asynchronous workflows, use:

```
Amazon SNS
Amazon SQS
```

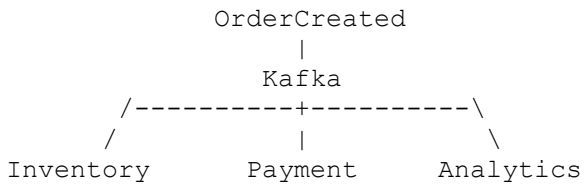
---

7. Publish-Subscribe

Publish:

```
OrderCreated
```

Multiple consumers subscribe:



Consumer:

```
@KafkaListener(
    topics = "ecommerce.orders",
    groupId = "inventory-service")
public void consume(OrderCreatedEvent event) {

    inventoryService.reserve(event);
}
```

Separate consumer groups when applications need independent delivery.

---

8. Event Notification vs Event-Carried State Transfer

Event Notification

Publish:

```
{
  "type": "ProductPriceChanged",
  "productId": "P100"
}
```

Consumer subsequently asks Catalog/Pricing Service for the current state.

Event-Carried State Transfer

Publish:

```
{
  "type": "ProductPriceChanged",
  "productId": "P100",
  "price": 79999,
  "currency": "INR",
  "version": 17
}
```

The consumer can update its local read model without another synchronous call.

Use schema/version management because event contracts become enterprise integration contracts.

---

9. Service Discovery

The attached document lists client-side, server-side, registry and DNS-based discovery, including Eureka, Consul and Kubernetes discovery. [List od Mi...iples.docx | Word]

For this Kubernetes solution, use:

```
Kubernetes Service + DNS
```

Create:

```
apiVersion: v1
kind: Service

metadata:
  name: inventory-service

spec:
  selector:
    app: inventory-service

  ports:
    - port: 80
      targetPort: 8080
```

Order Service calls:

```
http://inventory-service/api/inventory/P100
```

Kubernetes resolves:

```
inventory-service
      |
Cluster DNS
      |
```

```
Service
     |
Healthy Pods
```

This generally removes the need to add Eureka merely for service discovery inside a Kubernetes-native deployment.

---

10. Database per Service

Database-per-Service is explicitly included in the document. [List od Mi...iples.docx | Word]

Implement:

```
Catalog Service -------- Catalog DB
Order Service ---------- Order DB
Payment Service -------- Payment DB
Inventory Service ------ Inventory DB
Customer Service ------- Customer DB
```

Repository:

```
public interface OrderRepository
       extends JpaRepository<Order, UUID> {

    List<Order> findByCustomerId(UUID customerId);
}
```

Rule:

```
Payment Service
     X
     |
Direct SQL
     |
Order Database
```

Instead:

```
Payment -> Order REST API

or

OrderCreated event -> Payment
```

---

11. Polyglot Persistence

Choose persistence according to workload:

```
Orders      -> PostgreSQL
Payments    -> PostgreSQL
Catalog     -> DynamoDB/PostgreSQL
Cart        -> Redis
Search      -> Amazon OpenSearch
Documents   -> Amazon S3
Events      -> Amazon MSK
```

Do not introduce multiple database technologies merely because microservices theoretically permit it. Each technology adds operations, backup, security and monitoring responsibilities.

---

12. Transactional Outbox Pattern

This is one of the most important enterprise patterns explicitly highlighted in the document. [List od Mi...iples.docx | Word]

The problem:

```
orderRepository.save(order);
kafka.send(event);
```

Database succeeds:

```
Order = CREATED
```

but Kafka publication fails.

Now other services never receive `OrderCreated`.

Correct implementation

Create:

```
CREATE TABLE outbox_event (
    id UUID PRIMARY KEY,
    aggregate_id UUID NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMP NOT NULL,
    published BOOLEAN NOT NULL
);
```

Persist both records within one transaction:

```
@Transactional
public Order createOrder(CreateOrderCommand command) {

    Order order = Order.create(command);

    orderRepository.save(order);

    outboxRepository.save(
        OutboxEvent.from(
            order,
            "OrderCreated"
        )
    );

    return order;
}
```

Result:

```
TRANSACTION
   |
   +-- INSERT orders
   |
   +-- INSERT outbox_event
   |
 COMMIT
```

Then:

```
Outbox -> CDC/Publisher -> Kafka/MSK
```

A robust enterprise option is:

```
PostgreSQL
   |
Debezium CDC
   |
Kafka Connect
   |
Amazon MSK
```

---

13. Saga Pattern

Saga and its Orchestration/Choreography variants are explicitly listed in List od Microservices Design Patterns and Principles.docx. [List od Mi...iples.docx | Word]

Checkout Saga

```
1. Create Order
2. Reserve Inventory
```

```
3. Authorize Payment
4. Create Shipment
5. Confirm Order
```

Success:

```
PENDING
   |
InventoryReserved
   |
PaymentAuthorized
   |
ShipmentCreated
   |
CONFIRMED
```

Failure:

```
PaymentFailed
      |
ReleaseInventory
      |
CancelOrder
```

---

14. Saga Orchestration

Create:

```
@Service
class CheckoutSaga {

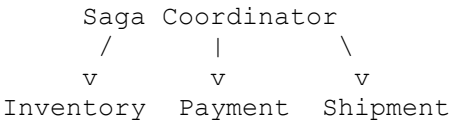
    public void start(UUID orderId) {
        commandBus.send(
             new ReserveInventoryCommand(orderId));
    }

    public void inventoryReserved(UUID orderId) {
        commandBus.send(
             new AuthorizePaymentCommand(orderId));
    }

    public void paymentFailed(UUID orderId) {
        commandBus.send(
             new ReleaseInventoryCommand(orderId));

        commandBus.send(
             new CancelOrderCommand(orderId));
    }
}
```

Architecture:

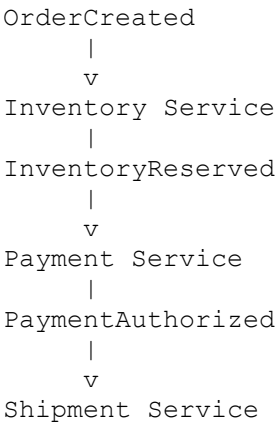


Choose orchestration when the checkout workflow needs central visibility and explicit workflow state.

---

15. Saga Choreography

Remove the coordinator:



Each service reacts independently.

Good for simpler event flows.

Be cautious when choreography grows into dozens of interconnected events because ownership and debugging become difficult.

---

16. Compensation Pattern

Define explicit reverse operations.

Forward operation	Compensation
Reserve inventory	Release inventory
Authorize payment	Void authorization

Capture payment	Refund payment
Create shipment	Cancel shipment
Confirm order	Cancel order

Design compensation as a **business operation**, not database rollback.

---

17. TCC

For resources supporting explicit reservations:

```
TRY
 |
Reserve 1 Laptop
 |
Authorize ₹79,999
 |
   +----------+
   |          |
Confirm    Cancel
```

Typical methods:

```
tryReservation()
confirmReservation()
cancelReservation()
```

Use only when participating services genuinely support reservation semantics.

---

18. CQRS

CQRS is explicitly listed under communication/data patterns. [List od Mi...iples.docx | Word]

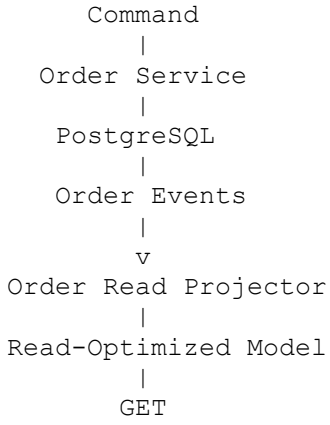
Separate commands:

```
POST /api/orders
PUT  /api/orders/{id}/cancel
```

from queries:

```
GET /api/order-view/{id}
GET /api/customers/{id}/order-history
```

Architecture:



Do not use CQRS automatically for simple CRUD services.

---

19. Event Sourcing

Instead of storing:

```
Order:
STATUS = SHIPPED
```

store:

```
OrderCreated
InventoryReserved
PaymentAuthorized
OrderConfirmed
OrderPacked
OrderShipped
```

Rebuild:

```
Order order = new Order();

events.forEach(order::apply);
```

Use Event Sourcing only where historical reconstruction/auditing and domain complexity justify the infrastructure and operational cost.

---

20. Materialized View

Build a denormalized customer-order view:

```
CustomerOrderView

Order ID
Date
Total
Payment Status
Shipping Status
Tracking Number
```

Projection consumer:

```
@KafkaListener(topics = "order-events")
public void project(OrderEvent event) {
    orderViewRepository.apply(event);
}
```

This prevents runtime joins across multiple microservice databases.

---

21. Cache-Aside

Caching patterns including Cache-Aside, Read-Through, Write-Through, Write-Behind and Distributed Cache are listed in the document. [List od Mi...iples.docx | Word]

Use Redis for Product Catalog.

```
@Cacheable(
    value = "products",
    key = "#productId")
public ProductDto findProduct(UUID productId) {
    return repository.findById(productId)
        .map(mapper::toDto)
        .orElseThrow(ProductNotFoundException::new);
}
```

Flow:

```
Request
   |
Redis
 /   \
Hit   Miss
 |      |
Return   PostgreSQL
           |
       Redis update
           |
         Return
```

Add TTL and explicit invalidation.

---

22. Circuit Breaker

Resilience patterns, including Circuit Breaker, Retry, Timeout, Bulkhead and Fallback, appear in the attached list. [List od Mi...iples.docx | Word]

Use Resilience4j:

```
@CircuitBreaker(
    name = "inventory",
    fallbackMethod = "inventoryFallback")
public InventoryResponse getInventory(String sku) {
    return inventoryClient.getInventory(sku);
}

private InventoryResponse inventoryFallback(
        String sku,
        Throwable exception) {

    return InventoryResponse.unavailable(sku);
}
```

State:

```
CLOSED
  |
Failures exceed threshold
  |
OPEN
  |
Wait
  |
HALF_OPEN
  |
Success
  |
CLOSED
```

---

23. Retry + Timeout

Configuration:

```
resilience4j:
  retry:
    instances:
      inventory:
```

```
        max-attempts: 3
        wait-duration: 200ms

  timelimiter:
    instances:
      inventory:
        timeout-duration: 2s
```

Recommended combination:

```
Timeout
   |
Retry transient failure
   |
Circuit Breaker
   |
Fallback / fail
```

Do not blindly retry:

```
POST payment capture
POST refund
POST create-order
```

unless idempotency is guaranteed.

---

24. Idempotency Principle

Although idempotency is not separately named in the attached list, it is essential when implementing retries, messaging, Saga and distributed transactions.

Client:

```
POST /api/orders
Idempotency-Key: 5f3d28...
```

Store:

```
Idempotency Key
Request Hash
Business Result
Expiry
```

On duplicate request:

```
Same key
   |
Do NOT create second order
```

```
   |
Return original result
```

For Kafka:

```
eventId -> processed_event table
```

Use a unique database constraint to prevent duplicate processing.

---

25. Bulkhead

Separate important resources:

```
Checkout Pool
     |
Payment

Catalog Pool
     |
Recommendations
```

Recommendation-service saturation must not consume every Order Service thread/connection.

Combine bulkheads with appropriate connection-pool and concurrency limits.

---

26. Rate Limiting and Throttling

Implement at:

```
AWS API Gateway
or
Spring Cloud Gateway
```

Logical policy:

```
Login       -> strict
Checkout    -> protected
Product GET -> high capacity
Admin APIs  -> restricted
```

Identity-aware keys can include:

```
Customer
API client
```

```
JWT subject
IP address
```

Return HTTP:

```
429 Too Many Requests
```

when limits are exceeded.

---

27. Dead Letter Queue

The DLQ pattern is explicitly included in the document's resilience section. [List od Mi...iples.docx | Word]

Flow:

```
Kafka/SQS
   |
Consumer
   |
Failure
   |
Retry
   |
Retry exhausted
   |
DLQ
```

Store metadata:

```
eventId
eventType
payload
failureReason
originalTopic
timestamp
retryCount
```

Provide:

```
DLQ monitoring
Alarm
Investigation
Controlled replay
```

Never silently discard failed checkout/payment events.

---

28. Health Checks

Add:

```
spring-boot-starter-actuator
```

Expose:

```
/actuator/health
```

Kubernetes:

```
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
```

Purpose:

```
Readiness -> Should receive traffic?

Liveness  -> Should Kubernetes restart it?
```

---

29. Graceful Degradation

Classify services:

```
Checkout        CRITICAL
Payment         CRITICAL
Inventory       CRITICAL

Recommendations NON-CRITICAL
Reviews         NON-CRITICAL
Analytics       NON-CRITICAL
```

Therefore:

```
Recommendation down
        |
Show popular products
        |
Checkout continues
```

But never return fake success for failed payment authorization.

---

30. Centralized Logging + Correlation ID

Observability patterns in the document include logging, tracing, correlation IDs, metrics, health endpoints, auditing and OpenTelemetry. [List od Mi...iples.docx | Word]

Generate:

```
X-Correlation-ID
```

at Gateway.

Propagate:

```
Gateway
   |
CID=ABC123
   |
Order
   |
Payment
   |
Inventory
```

Log structured JSON:

```
{
  "service": "order-service",
  "correlationId": "ABC123",
  "orderId": "O100",
  "event": "ORDER_CREATED"
}
```

On AWS, centralize logs in **CloudWatch Logs**.

---

31. Distributed Tracing

Instrument services with OpenTelemetry.

```
Trace
|
+-- Gateway span
+-- Order span
```

```
|    +-- PostgreSQL span
|
+-- Inventory span
|
+-- Payment span
```

Trace context should propagate across:

```
HTTP
Kafka
SQS
```

This allows a single failed checkout to be followed end-to-end.

---

32. Metrics

With:

```
Spring Boot Actuator
Micrometer
Prometheus-compatible metrics
```

Monitor the golden operational signals:

```
Request rate
Error rate
Latency
Saturation
```

Plus:

```
JVM heap
GC pauses
Database pool
Kafka lag
DLQ count
Payment failure rate
Order throughput
Circuit-breaker state
```

---

33. Audit Logging

Maintain an immutable business audit trail for sensitive actions:

```
Refund issued
Order manually cancelled
```

```
Customer privileges changed
Payment state modified
Admin operation performed
```

Record:

```
Actor
Action
Resource
Timestamp
Correlation ID
Previous State
New State
Result
```

Keep audit data distinct from ordinary debug logging.

---

34. OAuth2 + JWT + Gateway Security

Security patterns in the source include API Gateway security, OAuth2, JWT, Zero Trust, mTLS, service identity, secret management and token propagation. [List od Mi...iples.docx | Word]

Architecture:

```
Customer
    |
OAuth2/OIDC
    |
Identity Provider
    |
JWT
    |
Gateway
    |
Order Service
```

Spring Boot Resource Server:

```
@Bean
SecurityFilterChain security(HttpSecurity http)
        throws Exception {

    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health/**").permitAll()
            .requestMatchers("/api/orders/**").authenticated()
            .anyRequest().denyAll())
        .oauth2ResourceServer(oauth ->
            oauth.jwt(Customizer.withDefaults()))
```

```
        .build();
}
```

Do not trust traffic solely because it originates within the Kubernetes cluster.

---

35. Secrets + Workload Identity

Never package:

```
Database password
AWS access key
JWT signing key
Payment gateway secret
```

into:

```
Git
application.yml
Docker image
```

Use AWS services such as:

```
AWS Secrets Manager
AWS Systems Manager Parameter Store
IAM Roles for Service Accounts / workload identity
```

Architecture:

```
Order Pod
   |
Workload Identity
   |
IAM
   |
AWS Resource
```

Prefer short-lived workload credentials over hard-coded permanent keys.

---

36. Service Mesh + mTLS

Infrastructure patterns in the document include sidecar, ambassador, adapter and Service Mesh. [List od Mi...iples.docx | Word]

Possible Kubernetes implementation:

```
Istio / Linkerd
```

Use the mesh for:

```
mTLS
Traffic policy
Service identity
Telemetry
Traffic splitting
```

Traditional:

```
Pod
+----------------+
| Spring Boot    |
| Proxy Sidecar  |
+----------------+
```

The attached document also identifies emerging **sidecar-less service-mesh approaches**. [List od Mi...iples.docx | Word]

---

37. Adapter + Anti-Corruption Layer

Suppose E-Commerce must integrate with an old ERP:

```
Order Domain
    |
Anti-Corruption Layer
    |
ERP Adapter
    |
Legacy SOAP/XML ERP
```

Adapter:

```
@Component
public class LegacyErpAdapter
       implements ShipmentPort {

    private final LegacySoapClient client;

    public Shipment createShipment(Order order) {

        LegacyShipmentRequest request =
            mapper.toLegacy(order);

        LegacyShipmentResponse response =
            client.createShipment(request);
```

```
        return mapper.toDomain(response);
    }
}
```

The domain never depends directly on legacy ERP objects.

---

38. External / Distributed Configuration

Store environment configuration externally:

```
application
     |
ConfigMap
AWS Parameter Store
Secrets Manager
```

Example Kubernetes:

```
env:
  - name: ORDER_TOPIC
    valueFrom:
      configMapKeyRef:
        name: order-config
        key: order-topic
```

Separate:

```
Code
Configuration
Secrets
```

---

39. Docker: Immutable Deployment

Deployment patterns in the document include immutable, rolling, canary and blue-green deployments. [List od Mi...iples.docx | Word]

Dockerfile:

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/order-service.jar app.jar

USER 10001
```

```
ENTRYPOINT ["java","-jar","app.jar"]
```

Build:

```
order-service:2.4.7
```

Do not modify a running container.

Change:

```
Code
  |
CI build
  |
New image
  |
New deployment
```

---

40. Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: order-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: order-service

  template:
    metadata:
      labels:
        app: order-service

    spec:
      containers:
        - name: order-service
          image: ecommerce/order-service:2.4.7

          ports:
            - containerPort: 8080

          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
```

On AWS:

```
Docker Image
     |
Amazon ECR
     |
Amazon EKS
     |
Deployment
     |
Pods
```

---

41. Rolling Deployment

Kubernetes performs:

```
V1 V1 V1
   ↓
V1 V1 V2
   ↓
V1 V2 V2
   ↓
V2 V2 V2
```

Combine with readiness probes and backward-compatible interfaces/database migrations.

---

42. Canary Deployment

Deploy:

```
95% -> Version 1
 5% -> Version 2
```

Observe:

```
Error rate
P95/P99 latency
CPU/memory
Business KPI
Payment failures
```

Then gradually move traffic:

```
5%
25%
50%
```

```
100%
```

Rollback if key indicators regress.

---

43. Blue-Green Deployment

Maintain:

```
BLUE  -> Production V1
GREEN -> Candidate V2
```

Test GREEN.

Switch:

```
Traffic
BLUE  X
GREEN ✓
```

Retain BLUE temporarily for quick rollback.

---

44. Feature Toggle

Do not tie every feature activation to a deployment.

```
if (featureFlags.newCheckoutEnabled(customerId)) {
    return newCheckout.process(order);
}

return oldCheckout.process(order);
```

Use flags for gradual rollout, experimentation and temporary operational controls.

Remove stale flags after rollout to avoid permanent conditional complexity.

---

45. GitOps

The document explicitly includes GitOps under cloud-native/Kubernetes patterns. [List od Mi...iples.docx | Word]

Pipeline:

```
Developer
    |
Git Commit
    |
CI
    |
Tests
Security Scans
Build
    |
Docker Image -> ECR
    |
Update deployment manifest
    |
Git
    |
Argo CD / Flux
    |
Amazon EKS
```

Git becomes the desired deployment-state record.

---

46. Operator Pattern

For complex stateful platforms:

```
Custom Resource
      |
Kubernetes Operator
      |
Observe
      |
Reconcile
      |
Desired State
```

Typical candidates include Kafka/database/platform infrastructure rather than ordinary stateless CRUD microservices.

---

47. Leader Election

For a singleton activity:

```
Order Reconciliation Job
```

You might run:

```
Pod 1
Pod 2
Pod 3
```

but elect:

```
Pod 2 -> LEADER -> execute scheduler
```

If Pod 2 fails, another instance takes leadership.

Prefer idempotent jobs even when leader election exists.

---

48. Workflow Engine / Process Manager

The attached document lists orchestration, choreography, workflow engines, process managers and long-running transactions. [List od Mi...iples.docx | Word]

For a process such as:

```
Place Order
   |
Wait for Payment
   |
Reserve Inventory
   |
Fraud Check
   |
Wait for Warehouse
   |
Ship
   |
Wait for Delivery
```

persist workflow state rather than keeping Java threads/database transactions open.

Logical state:

```
ORDER_CREATED
PAYMENT_PENDING
PAYMENT_COMPLETED
INVENTORY_RESERVED
FULFILMENT_STARTED
SHIPPED
DELIVERED
```

A workflow engine becomes valuable when there are many timers, retries, human activities and compensations.

---

49. Long-Running Transaction

Never implement an hours/days-long order process as:

```
@Transactional
public void fulfilOrder() {
   // transaction remains open for hours
}
```

Instead:

```
Local TX
  |
Event
  |
Persist workflow state
  |
Local TX
  |
Event
```

This is essentially:

```
Saga + durable state + events + compensation
```

---

50. Cell-Based Architecture

Cell-based architecture appears in the cloud-native category of the document. [List od Mi...iples.docx | Word]

At very high scale:

```
Global Gateway
      |
+-----+-----+-----+
|           |     |
Cell A    Cell B Cell C
```

Each cell contains approximately:

```
Order
Cart
```

```
Inventory access
Payment integration
Databases/caches
Messaging
```

Partition customers/tenants geographically or logically.

Failure becomes:

```
Cell B failure
      X
Cell A and C continue
```

This significantly reduces blast radius for very large platforms.

---

51. Platform Engineering / IDP

The document also lists Platform Engineering and Internal Developer Platforms. [List od Mi...iples.docx | Word]

Provide developers a golden path:

```
Create New Microservice
        |
        v
Java/Spring Boot Template
        |
        +-- Security
        +-- Logging
        +-- OpenTelemetry
        +-- Actuator
        +-- Dockerfile
        +-- Helm chart
        +-- CI pipeline
        +-- Kubernetes policies
        +-- Dashboard
        +-- Alerts
```

This avoids 100 Spring Boot teams independently implementing security, health checks and telemetry differently.

---

52. Emerging Patterns

The attached document additionally lists AI Gateway, Event Streaming, Data Mesh Integration, AIOps, Edge AI and Agent-Based Services. [List od Mi...iples.docx | Word]

For this platform, treat these as extensions rather than mandatory checkout architecture.

Event Streaming

```
Orders -------\
Payments ------> Kafka/MSK -> Fraud
Inventory ----/            -> Analytics
                           -> Search
                           -> Recommendations
```

AI Gateway

```
E-Commerce Service
        |
    AI Gateway
        |
+------+-------+
|              |
Recommendation LLM/AI
Model          Services
```

Centralize authentication, policy, model routing, telemetry and usage controls.

Data Mesh

Publish domain-controlled analytical data products:

```
Order Domain     -> Order Data Product
Catalog Domain   -> Catalog Data Product
Customer Domain  -> Customer Data Product
```

AIOps

Apply anomaly detection to:

```
Metrics
Logs
Traces
Kafka lag
Pod health
Latency
Error spikes
```

AI/agent-based services should not bypass authorization, validation, workflow controls or audit requirements for business-sensitive operations.

---

53. Recommended Implementation Order

Do **not** attempt to implement every pattern simultaneously.

For this enterprise e-commerce project, implement them in this sequence:

```
PHASE 1: DOMAIN
   |
   +-- Business Capability
   +-- DDD
   +-- Bounded Context
   +-- Aggregate
   +-- Database per Service

PHASE 2: BASIC MICROSERVICES
   |
   +-- Spring Boot
   +-- REST API
   +-- Spring Data JPA
   +-- Hibernate
   +-- PostgreSQL
   +-- API Gateway

PHASE 3: DISTRIBUTED COMMUNICATION
   |
   +-- Kafka / Amazon MSK
   +-- Domain Events
   +-- Pub/Sub
   +-- Transactional Outbox
   +-- CDC

PHASE 4: BUSINESS CONSISTENCY
   |
   +-- Saga
   +-- Compensation
   +-- Idempotency
   +-- CQRS where justified

PHASE 5: RESILIENCE
   |
   +-- Timeout
   +-- Retry
   +-- Circuit Breaker
   +-- Bulkhead
   +-- DLQ
   +-- Rate Limiting

PHASE 6: SECURITY
   |
   +-- OAuth2/OIDC
   +-- JWT
   +-- Secrets Manager
   +-- Workload Identity
   +-- mTLS where justified
```

```
PHASE 7: OBSERVABILITY
   |
   +-- Structured Logs
   +-- Correlation IDs
   +-- Micrometer Metrics
   +-- OpenTelemetry
   +-- Distributed Tracing
   +-- Audit Trail

PHASE 8: CLOUD-NATIVE DEPLOYMENT
   |
   +-- Docker
   +-- ECR
   +-- Kubernetes/EKS
   +-- Service Discovery
   +-- Health Probes
   +-- Rolling/Canary

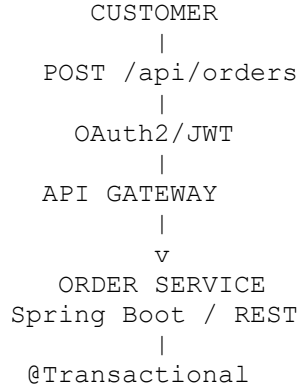
PHASE 9: PLATFORM AUTOMATION
   |
   +-- GitOps
   +-- Argo CD/Flux
   +-- Operators
   +-- Platform Engineering

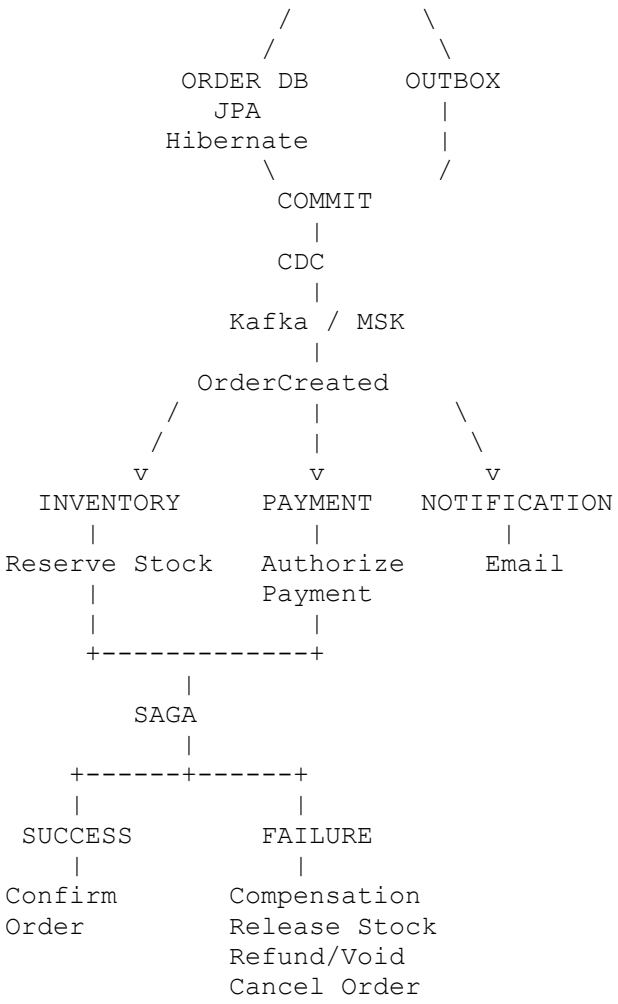
PHASE 10: ADVANCED SCALE
   |
   +-- CQRS
   +-- Event Sourcing where justified
   +-- Service Mesh
   +-- Multi-cluster
   +-- Cell Architecture
```

---

54. Complete Enterprise Order Implementation

This is the most important flow to remember for interviews and actual implementation:





Wrap the complete architecture with:

```
                  CROSS-CUTTING
-------------------------------------------------
Security        OAuth2, JWT, Zero Trust, mTLS
Resilience      Timeout, Retry, Circuit Breaker
Data            JPA, Hibernate, DB per Service
Caching         Redis
Messaging       Kafka/MSK, SNS/SQS
Reliability     Outbox, Idempotency, DLQ
Observability   Actuator, Micrometer, OpenTelemetry
Configuration   Parameter Store, Secrets Manager
Packaging       Docker
Runtime         Kubernetes / Amazon EKS
Images          Amazon ECR
Deployment      Rolling / Canary / Blue-Green
Delivery        CI/CD + GitOps
-------------------------------------------------
```

The interview-quality summary

I would first decompose the e-commerce domain using DDD and bounded contexts into independently deployable Spring Boot services such as Order, Catalog, Inventory, Payment and Shipping. Each service owns its data through Spring Data JPA/Hibernate and Database-per-Service. REST is used where an immediate response is required, while Kafka/Amazon MSK events are used for asynchronous workflows. Transactional Outbox prevents the database-message dual-write problem, Saga with compensating transactions handles distributed checkout consistency, and idempotency protects retries and duplicate message delivery. Resilience4j provides timeout, retry, circuit breaker and bulkhead protection. OAuth2/JWT and workload identity secure the platform, OpenTelemetry/Micrometer/Actuator provide observability, and the services are packaged as immutable Docker images and deployed to Amazon EKS with Kubernetes service discovery, health probes and rolling/canary delivery through a GitOps pipeline.

That implementation narrative directly connects the highest-priority patterns identified in List od Microservices Design Patterns and Principles.docx to a realistic enterprise Java/Spring Boot e-commerce architecture. [List od Mi...iples.docx | Word]